

ActiveEnv

The agent that catches the config value that is present, valid, authenticating — and still silently wrong.

Hackathon	Alibaba Cloud / Qwen Global AI Hackathon
Track	Track 4 — Autopilot Agent
Submission deadline	9 July 2026
Build mode	Solo · target 2 weeks · \$0 actual spend
Core stack	Next.js · Django REST · Celery/Redis · Qwen (Model Studio) · custom MCP · Alibaba Cloud ECS
Status	Scope locked (\$10 + \$17) — nothing built yet; this is the target

Reading order: Introduction → Problem → What it's for → Who it affects → Why (the moat) → How it's built → The phased roadmap → Compliance & risk.

What ActiveEnv is

ActiveEnv is an Autopilot agent that catches the configuration value that *looks* correct but is silently wrong. It does this by inferring what each value is *supposed* to be from how it is used in the codebase, then **actively probing the real system** to see what the value *actually does* — flagging mismatches with evidence, proposing a fix, and (after human approval) applying it and re-probing until reality matches intent.

It is **not** a .env linter. A linter checks *format and presence*. ActiveEnv checks **truth**: it connects, authenticates, and observes real systems to catch the error a linter physically cannot see — the value that is present, valid-looking, and pointing at the wrong target.

The closed loop (the product's signature moment)

```
infer intent → probe reality → classify → human approves → apply fix → re-probe → green
```

The loop closing on screen — fix applied, agent re-probes, the row turns green — is the demo's payoff and the single most important second of the submission.

02 · THE PROBLEM

The bug that passes every check

Every developer has shipped a config that passed every check and still broke production — not because a variable was missing or malformed, but because it was present, valid, and silently pointing at the wrong target. Missing or typo'd variables are easy: the app crashes immediately and you fix it. The expensive failures are the **silent** ones — config that boots cleanly, passes CI, looks perfect in review, and quietly does the wrong thing for hours or weeks before anyone notices.

Variable	Looks fine because...	What is actually wrong	Cost
STRIPE_SECRET_KEY	Authenticates successfully	A test -mode key in the production config (or a live key in test)	Lost revenue / real money moved
DATABASE_URL	Connects, app boots	Points at the staging DB while the app believes it is prod	Data corruption
GITHUB_TOKEN	Authenticates, calls succeed	Wrong scope or wrong account/org than the code assumes	Broken CI, misdirected access

03 · WHAT IT'S FOR

Purpose & locked v1 scope

ActiveEnv exists to do one thing extremely well: **prove, with live evidence, whether each config value points where the code expects**. v1 is deliberately narrowed to three adapters that are each *provably* reliable — depth on three real probes beats breadth across many unverifiable ones.

v1 adapter	The silent-wrong it catches	Why it is reliable
Postgres DB probe	Staging DB used in prod	SELECT current_database(), host, version genuinely reveals which DB answered.

v1 adapter	The silent-wrong it catches	Why it is reliable
Stripe key probe	Test key in prod (or live in test)	Keys are self-describing (sk_test_/sk_live_) and the API exposes a livemode flag.
GitHub token probe	Wrong account / org / scope	GET /user and /repos cleanly reveal identity, org, and scope.

Explicitly out of v1 (roadmap only): the generic “which environment answered this URL” probe (no reliable universal signal), VS Code / CI extensions, full multi-language support, and auto-fix without approval. Guarding this scope is non-negotiable for a 2-week solo build.

04 · WHO IT AFFECTS

Audience & reach

- **Primary:** solo developers, small teams, and indie/SaaS builders deploying across dev / staging / prod *without* a dedicated DevOps or SRE team to catch config drift.
- **Secondary:** any backend or full-stack engineer doing a deploy, an environment migration, or a new service onboarding.
- **Scale story:** every CI pipeline and pre-deploy checklist is a distribution channel. The pain is universal and largely framework-agnostic — which is exactly why it scores on impact.

05 · WHY IT WINS

The differentiator — three things a linter cannot do

Capability	Why a linter cannot replicate it
Infer intent from usage	The agent reads the code to learn a key is used on a customer-charging path, so in a prod config it must be <i>live</i> . That is LLM reasoning over the codebase — not pattern-matching a string.
Probe reality, don't read values	It does not trust the string. It uses the credential against the real system and observes the truth (DB identity query, provider identity/mode endpoint).
Detect present-but-wrong	Missing or malformed is trivially caught by everyone. ActiveEnv is judged entirely on catching the value that is present, valid, authenticating — and still wrong, with evidence.

Pipeline, architecture & stack

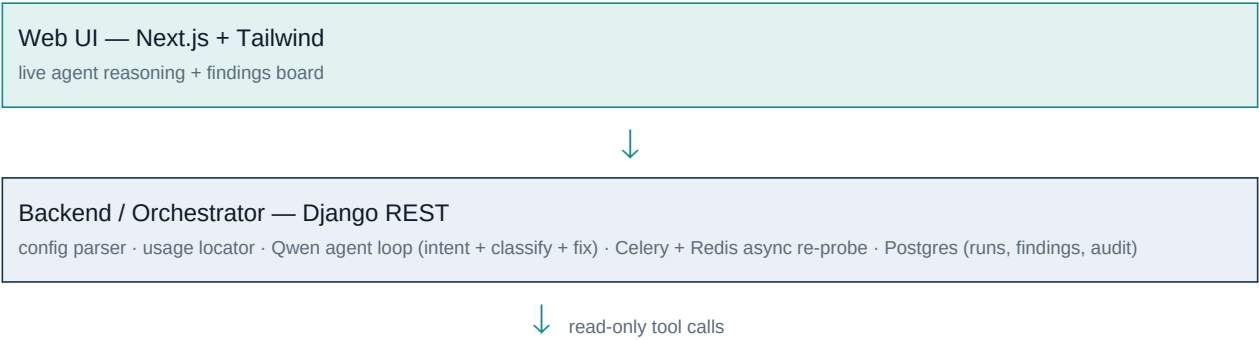
The 11-step pipeline (per run)

#	Stage	What happens
1	Ingest	User provides config (.env / settings.py / yaml / json) + codebase (paste, upload, or repo URL). Secrets masked immediately.
2	Inventory & locate	Parse every config key; find <i>where and how</i> each is used in the code.
3	Infer intent (Qwen)	Expected type, expected environment, what it gates, expected properties — from the usage slice, not the whole repo.
4	Plan probes	Select the right adapter (DB / Stripe / GitHub) per key.
5	Execute probes (MCP)	Connect, authenticate, or GET via read-only MCP tools. Never write.
6	Compare & classify	Correct · suspect · silently-wrong.
7	Explain with evidence	What was expected, what reality showed, the proof, the blast radius.
8	Propose fix (Qwen)	Corrected value/target with a one-line rationale.
9	Human-in-the-loop gate	User reviews and approves. Nothing changes without approval.
10	Apply + re-probe	Apply the fix, re-run the probe to confirm green.
11	Audit log	Every inference, probe, result, confidence, fix, and undo recorded.

Two seams we hardened beyond the original brief

- **Bounded ingestion (retrieval, not whole-repo):** intent is inferred from the *usage sites* of each key (AST / reference scan → surrounding function), so context stays small and inference stays grounded even on large multi-file codebases.
- **Named MCP boundary:** Qwen decides *what* to probe via native function-calling; the Django orchestrator routes each tool call to the `config-probe-mcp` server, which is the single place that enforces read-only execution, secret-masking, and the audit log.

Architecture (data flow)



MCP Server — config-probe-mcp

probe_database · probe_stripe_key · probe_github_token



Real systems — Postgres/MySQL · Stripe · GitHub

All backend components deploy to Alibaba Cloud ECS (Singapore) via Docker. Frontend may host on Vercel — only the backend must be on Alibaba Cloud, per hackathon rules.

Tech stack

Layer	Choice	Role
Frontend	Next.js (React) + Tailwind	The demo surface: live-streamed reasoning + findings board
Backend	Python · Django REST Framework	Builder's home stack — fastest reliable path
Async / scheduling	Celery + Redis	Probes run async; enables the 're-probe later' beat
LLM	Qwen via Model Studio (OpenAI-compatible)	qwen-max / qwen-plus, function calling / tool use
Agent ↔ tools	MCP server (3 probe tools)	The rubric's named 'MCP integration' — primary innovation hook
Datastore	Postgres	Runs, findings, audit log
Probes	psycopg / pymysql, httpx, Stripe SDK, GitHub REST	All strictly read-only
Deploy	Alibaba Cloud ECS (Singapore) + Docker	Mandatory; ship a code file proving Alibaba Cloud API use

Phased roadmap — small steps, in order

Nine phases, each a small shippable increment. We build top to bottom and do not start a phase until you say so. Phase 0 clears the pass/fail compliance gates *first*, so nothing can disqualify us late. The demo-critical phases (3–6) get the most time.

PHASE 0	Foundations & compliance gates	Day 1–2
Goal	Stand up the skeleton and clear every pass/fail hackathon gate before writing product code.	
Steps	• Create public GitHub repo + add an OSS license (visible in About) + README skeleton. • Activate Alibaba Cloud account; provision the free-tier ECS instance (Singapore). • Get Qwen Model Studio API key; make one successful qwen call and commit it as the Alibaba-proof code file. • Scaffold local dev: Django REST + Postgres + Redis + a Next.js app that talk to each other.	
Done when	✓ Repo is public & licensed, ECS is reachable, a real Qwen call returns, all four services run locally.	

PHASE 1	Config ingestion & inventory	Day 3–4
Goal	Turn raw config + codebase into a structured, secret-masked inventory of keys and their usage.	
Steps	• Config parser for .env / settings.py / yaml / json; mask secrets the moment they are read. • Codebase usage locator: scan for env references (os.environ / settings.X / getenv) and capture each usage site + surrounding function. • Data models: Run, ConfigKey, UsageSite, Finding, AuditEntry. • API endpoint: submit a run (paste/upload/repo URL) → returns the parsed, masked inventory.	
Done when	✓ Submitting a sample config + repo yields a masked key list, each linked to its real usage sites.	

PHASE 2	Intent inference (Qwen)	Day 4–5
Goal	For each key, infer what it is supposed to be — from its usage slice, not the whole repo.	
Steps	• Build the per-key retrieval slice (usage sites + context) so prompts stay within context limits. • Design the Qwen intent prompt + a strict structured-output schema (expected type, env, what it gates, expected properties). • Wire qwen-max/plus via Model Studio; store inferred intent + a confidence score on each key. • Stream the reasoning text so it can be shown live in the UI later.	
Done when	✓ Each key has a structured, grounded intent record (e.g. 'charges customers → prod must be live').	

PHASE 3	MCP probe server — the core innovation	Day 6–8
Goal	Build the config-probe-mcp server exposing three strictly read-only probe tools.	
Steps	• Scaffold the MCP server + tool schemas; enforce a hard read-only / no-write boundary. • probe_database: connect & run SELECT current_database(), host, version (psycopg/pymysql). • probe_stripe_key: detect sk_test_/sk_live_ and call the API for the livemode flag (Stripe SDK). • probe_github_token: GET /user and /repos to reveal identity, org, and scope (httpx). • Return normalized evidence objects; mask every secret in inputs, outputs, and logs.	
Done when	✓ Each tool, called in isolation, returns real evidence from a live system — and can never write.	
PHASE 4	Orchestration & classification	Day 8–10
Goal	Let Qwen drive the probes and turn intent-vs-reality into a classified finding with evidence.	
Steps	• Implement the Qwen function-calling agent loop in Django; map tools → MCP calls and feed results back. • Plan-probes step: pick the right adapter per key automatically. • Classifier: compare intent vs probe reality → correct / suspect / silently-wrong. • Assemble the evidence + blast-radius explanation for each finding.	
Done when	✓ A full run auto-probes every key and produces classified findings, each backed by live evidence.	
PHASE 5	Human-in-the-loop fix & the green loop	Day 10–11
Goal	Close the loop: propose a fix, gate on human approval, apply, re-probe, confirm green.	
Steps	• Qwen proposes a corrected value/target + one-line rationale per silently-wrong finding. • HITL approval gate: nothing is applied without explicit user approval. • Apply the approved fix, then trigger an async re-probe (Celery) and flip the row to green on success. • Full audit log with undo on every action.	
Done when	✓ Approving a fix visibly re-probes and turns the finding green; every action is logged & undoable.	
PHASE 6	Frontend — the demo surface	Day 11–13
Goal	Build the screen that makes the reasoning and the reveal visible — 15% of the grade lives here.	
Steps	• Run screen: paste/upload config + repo, hit Run. • Live panel: stream Qwen intent reasoning + visible MCP tool calls as they happen. • Findings board: correct / suspect / silently-wrong with evidence and blast radius. • Approve-fix interaction + the green re-probe animation; audit-log view with undo.	
Done when	✓ An end-to-end run is fully watchable on screen, ending in the green-loop payoff.	

PHASE 7	Deploy + compliance proof	Day 13
Goal	Ship the backend to Alibaba Cloud and lock down every required submission artifact.	
Steps	• Dockerize the backend; deploy to Alibaba Cloud ECS (Singapore); deploy frontend to Vercel. • Finalize the Alibaba-proof code file + confirm Qwen + MCP are wired in production. • Produce the architecture diagram + text description; complete the README. • Add the OSS license check and the Stage-1 compliance checklist to the repo.	
Done when	✓ The live app runs on Alibaba Cloud and every Stage-1 pass/fail item is checked off.	

PHASE 8	Demo data, video & submission	Day 14
Goal	Stage the two hero failures, rehearse, record under 3 minutes, and submit.	
Steps	• Seed demo data: staging + prod Postgres DBs, a test + live Stripe key, a scoped GitHub token. • Rehearse the ≤3-min script; dry-run the generate + re-probe beats until the green flip is rock-solid. • Record the demo video (English, public, <3 min) showing the test-key-in-prod & staging-DB-in-prod reveals. • Final pass over the submission checklist; submit. (Optional: build-journey blog post for bonus.)	
Done when	✓ Video is published, all deliverables are linked, and the submission is in before the deadline.	

08 · GUARDRAILS

Compliance gates, safety & the real risks

Stage-1 compliance (pass/fail — clear these or nothing else counts)

- Built with Qwen models on Qwen Cloud / Model Studio.
- Backend deployed on Alibaba Cloud + a code file proving Alibaba Cloud service/API use.
- Public repo with a detectable open-source license (visible in About).
- Architecture diagram + text description.
- Demo video under 3 minutes, public, in English. Track identified: Autopilot Agent.

Safety model (non-negotiable)

- Probes are strictly read-only — connect / authenticate / GET only. Never write against user systems.
- Secrets masked everywhere (UI, logs, audit). Never stored in plaintext or placed in URL params.
- Fixes require explicit human approval before being applied; re-probe after to confirm.
- Full audit trail with undo on every action — the ‘production-ready, not a toy’ proof.

Top risks & how the plan handles them

Risk	Type	Mitigation in this roadmap
The live ‘turns green’ re-probe is flaky on demo day	Execution	Phase 8 rehearses the beat until rock-solid; Phase 5 makes re-probe a first-class async step.
Alibaba / Qwen / MCP compliance missed late	Pass/fail	Cleared first in Phase 0; re-confirmed in Phase 7 before anything else.

Risk	Type	Mitigation in this roadmap
Large codebase blows the context window, intent inference degrades	Technical	Phase 1–2 use usage-site retrieval, not whole-repo ingestion — context stays bounded.
Scope creep (generic ‘any service’ probe)	Schedule	Explicitly out of v1; three provable adapters only. Guarded fiercely.

Status: scope locked, name locked (**ActiveEnv**), roadmap defined. Nothing is built yet. Next action is your go-ahead to begin **Phase 0** — and not before.